



Owning Outcomes

Community Experience Team

An introduction to Owning Outcomes

Ownership & Accountability

1. Take personal responsibility for outcomes, not just task completion.
2. Follows through on commitments without reminders.
3. Close loops and own mistakes without deflection.

Adaptive Problem-Solving

1. Navigate incomplete or unclear requirements without paralysis.
2. Ask clarifying questions to reduce ambiguity.
3. Pivot approach when new information emerges and makes progress with imperfect information.

Always on
collaboration

Open discussion

Why isn't collaboration
the default setting for
every team?

The Five Dysfunctions of a Team

Patrick Lencioni



Absence of trust



Fear of conflict



**Lack of
commitment**



**Avoidance of
accountability**



**Inattention to
results**

What's the real blocker?

Respect

Trust

Without both:

- Competition turns toxic
- Communication feels risky
- Clarity gets lost in hidden agendas

"Teams fail not because they lack skill or strategy, but because they lack trust — the foundation on which all else depends." -*Patrick Lencioni, author of The Five Dysfunctions*

Collaboration principles

Be open-minded



Value different perspectives and understand you can learn something new no matter how experienced you are. Always assume the best intentions.

Listen actively



Not just to reply, but to understand. Start all conversations from a place of curiosity.

Be accountable



Keep your word, and do what you'll say you'll do. If anything goes awry, communicate your blockers early.

Give credit freely



Celebrate others' wins, and give specific, timely, and relevant praise.

Disagree and commit



Don't take work personally, disagree on ideas without attacking the person.

Breakout rooms: Disagree and Commit

Your team is upgrading the client's internal search engine and must choose the technical approach.

The innovators

You strongly advocate for migrating to a brand-new Agentic AI framework. It is cutting-edge and promises a massive 50% performance gain, but the team has never used it and documentation is sparse.

The pragmatics

You strongly advocate for sticking with your proven, standard RAG pipeline. It is highly reliable, the team knows it perfectly, and it guarantees you will hit the sprint deadline, but it won't deliver flashy speed improvements.

The Task (10 Minutes):

- **Debate** (6 mins): Argue fiercely but respectfully for your assigned side.
- **Decide** (1 min): The group must vote and commit to only one path. No compromises or hybrid approaches allowed.
- **Commit** (3 mins): The faction whose idea was rejected must draft a public Slack message to the wider team demonstrating "Disagree and Commit."

Applying The Principles

Your colleague suggests a new tool or approach that you've never tried and you're skeptical it will work	✓	Let's explore your idea and maybe run a quick test to see if it improves our workflow.
	✗	That won't work. We've always done it this way.
Someone sends a long message explaining a challenge on Slack/Teams and you skimmed and think you have a general idea of the issue.	✓	I read through your message – here's what I understand, Did i get that right before we moved forward?
	✗	I already answered this, just follow the instructions I pinned
Your work has errors caused by subtle changes in the user story that affected the team's progress.	✓	I missed that update — I'll review the latest requirements now and correct it quickly.
	✗	I didn't know the requirement changed — someone should have told me.
A stakeholder praises the project outcome after a presentation, but the idea and most of the groundwork came from another teammate.	✓	Thank you! A lot of this success comes from MC's original concept and the team's effort in building it out
	✗	Thanks a Lot, I personally did my best to ensure this is a success.
The team chooses a different path than you suggested.	✓	I had a different view, but now that we've decided, I'm fully onboard — let's make it work.
	✗	Well, don't blame me when this fails.

From Clash to Collaboration: A 5-Step Peace Treaty

1. **Pause** and acknowledge the emotion
2. **Reaffirm** respect and intent
3. **Invite** Perspective
4. **Listen** actively, summarize and show understanding
5. **Close** with trust and appreciation.



Adaptive problem-solving

Conquering Ambiguity Paralysis

Vague request



Clarifying questions



Interim solution

~~Waiting for perfect requirements~~

The Ambiguity Breaker Framework:

- **Stop waiting:** Don't stall while waiting for perfect requirements.
- **Ask the right questions:** Always ask, "What do we know?" and "What don't we know?" before acting.
- **Bridge the gap:** Propose interim solutions or plan incremental deployments to balance experimentation with reliability.

PEAS framework

A method for resolving ambiguity from the Montana's State University School of Business.

1. Problem

Clearly identify and define the problem or question.

2. Evidence

Gather all relevant information, data, and facts related to the problem.

3. Analysis

Evaluate the evidence to understand its meaning and how it connects to the problem. Look for patterns, trends, and relationships.
Consider at least 3-4 theories or solutions, and evaluate the risks and implications of all.

4. Solution

Based on your analysis, choose the best solution to develop, and test its efficacy.

The Signal Card: Plan before you build

1. **Clarify the stakeholder and decision:** For a GitHub Enterprise rollout, identify who owns the go/no-go call and what metric or risk they care about.
2. **Define the problem and metrics:** Frame the key question and metric that matters. Example: 'How do we reduce licence provisioning latency from 24h to <2h?'
3. **Smallest proof in 48 hours:** Ship a quick spike, data chart, or simulation to show movement early, even if partial.
4. **Where to surface it:** Share updates where stakeholders make decisions: GitHub Issue, Slack thread, or Enterprise client portal.
5. **Evidence to collect:** Decide ahead: what number, quote, or artefact will show value? Capture these items before starting the task.

Ownership and *accountability*

Two Engineers, One PR

Engineer A:
minimal visibility

 **Dev dev**  5:11 PM
Hi team, the [PR](#) is ready 

Engineer
B: Outcome-Oriented
Communication

 **Dev dev**  5:11 PM
Hi team, the [PR](#) is ready 

- Reduced GitHub Actions build time by 27% (11m → 8m)
- Unblocks nightly releases
- Saves ~14 eng-hours/week
- Rolling out to two repos by Friday.

Observable Outcomes Drive Perception: In remote settings, output isn't an outcome. Engineer B's clarity and framing created an executive-readable brand moment.

Brand = Observable Evidence, Repeated: Reputation is built not by intent but by repeated, visible proof of impact. Engineer B created a lasting, promotable impression.

Owning the outcome



Communicate effectively

Articulates technical trade-offs in terms of business impact. Bring visibility to your work (use a Kanban, add subtasks, use playbacks, keep users informed).



Think critically

The ability to apply critical thinking to define, analyze, and solve unstructured problems. Manages ambiguity by defining a path forward on unclear tasks. Apply design thinking to understand why.



Take initiative

The drive to act proactively and move tasks forward, especially in the face of ambiguity, without constant guidance. Receives and incorporates feedback professionally. Pivots quickly based on new requirements or feedback



Signal high visibility

Articulates technical trade-offs in terms of business impact. Bring visibility to your work (use a Kanban, add subtasks, use playbacks, keep users informed).

Shift your vocabulary to show accountability

Your language dictates your brand, and can help make or break your reputation.

✗ "I'll try to get this done by Friday."

✓ Use Commitment Language: "I will have this completed by Friday."

✗ (Silence until the stand-up meeting)

✓ Unprompted Updates: Provide proactive status updates before the client has to ask.

✗ "The API was down, so I couldn't finish."

✓ Own the Blocker: Communicate blockers proactively and present a remediation plan.

✗ "I didn't know the requirement changed — someone should have told me."

✓ Owning It: "I missed that update — I'll review the latest requirements now and correct it quickly."

V

A

L

U

E messages

Value Created or Protected

In a GitHub Actions example: "Reduced build time 27% (11m→8m) saves ~14 engineer-hours/week across CI pipelines."

Alignment to Team or OKR

Tied the update to the Q3 goal: "Ship weekly without hotfixes." Shows strategic context beyond code.

Leverage Unlocked

Docs and QA teams can now move forward with Friday's rollout; the CI bottleneck is resolved.

Unblocker with Owner & Due Date

Asks clearly: "Need SRE to review cache key by Wed (tag @name)." Assigns ownership and urgency.

Evidence in Data or Artefacts

A chart and logs were shared in PR #1482. Updates include both visuals and an audit trail.

Chat discussion: Blocker re-write

Rewrite the update below to demonstrate ownership and accountability. You must use commitment language, acknowledge the error or blocker proactively, and present immediate remediation steps.

The bad update

“ I'll try to finish the ticket, but the third-party API has been down so I can't do anything right now.



Q&A

